

Test du Pipeline Data

Vérification end-to-end : Fact Table → Agrégats → Features RFM

CE QUE CE DOCUMENT PERMET DE VÉRIFIER

1. Les nouvelles transactions arrivent bien dans `fact_transaction`
2. La table `daily_sales` est recalculée correctement
3. La table `products_performance` reflète les ventes réelles
4. Le RFM d'un client (`customers_features_train`) est cohérent
5. Aucun client ni aucune transaction n'est perdu(e) en cours de pipeline
6. Le chiffre d'affaires total est identique entre les tables source et agrégées

Étape 1 — Vérifier que les nouvelles transactions existent dans la fact

Cette requête permet de confirmer que les transactions générées pour une date donnée sont bien présentes dans la table de faits.

```
SELECT
    d.date,
    COUNT(*) AS nb_transactions,
    SUM(f.price) AS chiffre_affaires
FROM fact_transaction f
JOIN dim_date d
    ON f.date_key = d.date_key
WHERE d.date = '2026-07-28'
GROUP BY d.date;
```

Étape 2 — Vérifier que daily_sales a été recalculée

Requête 1 : consulter la ligne agrégée du jour dans daily_sales.

```
SELECT *
FROM daily_sales
WHERE date_key = 20260728;
```

Requête 2 : recalculer manuellement à partir de la fact pour comparaison.

```
SELECT
    COUNT(*) AS nb_transactions,
    SUM(price) AS chiffre_affaires
FROM fact_transaction
WHERE date_key = 20260728;
```

Résultat attendu : les deux résultats (nb_transactions et chiffre_affaires) doivent être **identiques**.

Étape 3 — Vérifier products_performance

Articles testés : à choisir selon le fichier CSV généré à la date du jour (exemples ci-dessous).

778436001

877400003

870611001

788946001

694670008

Calculer le nombre réel de ventes dans la fact pour ces articles :

```
SELECT
  article_key,
  COUNT(*) AS ventes
FROM fact_transaction
WHERE article_key IN (
  778436001,
  877400003,
  870611001,
  788946001,
  694670008
)
GROUP BY article_key
ORDER BY article_key;
```

Puis comparer avec la table agrégée :

```
SELECT
  article_key,
  n_sales
FROM products_performance
WHERE article_key IN (
  778436001,
  877400003,
  870611001,
  788946001,
  694670008
)
ORDER BY article_key;
```

Résultat attendu : le champ n_sales doit être exactement le même que ventes, article par article.

Étape 4 — Vérifier le RFM d'un client

Exemple customer_id = **dfaf1056b112f0d796c0181d5cd6d5f9de7a781dcee597d2e3a6476e1a517128**

1. Retrouver sa clé interne (customer_key) :

```
SELECT
    customer_key
FROM dim_customer
WHERE customer_id = 'dfaf1056b112f0d796c0181d5cd6d5f9de7a781dcee597d2e3a6476e1a517128';
```

Supposons que la requête retourne la clé **214587963**.

2. Calculer les statistiques réelles à partir de la fact :

```
SELECT
    COUNT(*) AS nb_transactions,
    SUM(price) AS total_spend,
    MIN(date_key) AS premiere_date,
    MAX(date_key) AS derniere_date
FROM fact_transaction
WHERE customer_key = 214587963;
```

3. Comparer avec la table des features clients :

```
SELECT
    total_spend,
    n_transactions,
    first_purchase,
    last_purchase,
    recency_days,
    tenure_days
FROM customers_features_train
WHERE customer_key = 214587963;
```

Résultat attendu :

- total_spend = SUM(price)
- n_transactions = COUNT(*)
- last_purchase = 2026-07-28
- first_purchase = première date d'achat du client
- recency_days = 0 (si le pipeline est lancé le 28/07)

Étape 5 — Vérifier la dernière date d'achat des clients actifs aujourd'hui

Vérifier que tous les clients ayant acheté aujourd'hui ont bien une `last_purchase` = date du jour.

```
SELECT
  customer_key,
  last_purchase,
  recency_days
FROM customers_features_train
WHERE last_purchase = '2026-07-28'
LIMIT 20;
```

Résultat attendu : la liste retournée doit correspondre aux clients ayant réellement acheté dans le flux du jour.

Étape 6 — Vérifier qu'il n'y a pas de clients perdus

```
SELECT COUNT(*)
FROM dim_customer;
```

Comparer avec :

```
SELECT COUNT(*)
FROM customers_features_train;
```

Résultat attendu : les deux nombres doivent être **identiques**.

Étape 7 — Vérifier qu'aucune transaction n'est oubliée

```
SELECT COUNT(*)
FROM fact_transaction;
```

Comparer avec :

```
SELECT SUM(nb_transactions)
FROM daily_sales;
```

Résultat attendu : les deux valeurs doivent être **identiques**.

Étape 8 — Vérifier le chiffre d'affaires total

```
SELECT SUM(price)
FROM fact_transaction;
```

Comparer avec :

```
SELECT SUM(chiffre_affaires)
FROM daily_sales;
```

Résultat attendu : les deux sommes doivent être **identiques**.

Étape 9 — Vérifier products_performance (cohérence globale)

```
SELECT SUM(n_sales)
FROM products_performance;
```

Comparer avec :

```
SELECT COUNT(*)
FROM fact_transaction;
```

Résultat attendu : les deux valeurs doivent être **égales**, car chaque transaction correspond à la vente d'un seul article.

Guide de test du recalcul — Ingestion streaming

Requêtes SQL prêtes à copier-coller, regroupées par bloc AVANT et bloc APRÈS. Toute personne peut exécuter le bloc AVANT, injecter les nouvelles transactions, laisser tourner le recalcul, puis exécuter le bloc APRÈS et comparer les résultats.

À modifier avant d'utiliser ce document pour un nouveau test :

- La date testée (ex. 2026-07-21) dans toutes les requêtes concernées
- La liste des customer_id (nombre de clients variable selon le test)
- La liste des article_id concernés (table products_performance)
- Le CA attendu (somme des price des transactions injectées)
- Le nombre de transactions attendu

Paramètres de cet exemple

Paramètre	Valeur utilisée dans cet exemple	À adapter à chaque nouveau test
Date testée	2026-07-21	La date des nouvelles transactions injectées
Liste des customer_id (5)	82e136de..., 306b5096..., dfaf1056..., 554b1651..., 9fc68991...	Les customer_id réellement injectés
Liste des article_id	0740723001, 0656348001, 0603584008, 0614714004 (+ 5e à compléter)	Les article_id réellement injectés
CA attendu (somme des price)	0.090672	Recalculer la somme des price des transactions injectées
Nombre de transactions attendu	5	Le nombre exact de lignes injectées
Ancien(s) segment_valeur des clients	à relever avant injection	Segment(s) d'origine des clients concernés (voir 1.4)

BLOC 1 — REQUÊTES "AVANT" (à exécuter avant l'injection streaming)

À copier-coller et exécuter intégralement avant d'injecter les nouvelles transactions. Conserver les résultats (copie ou capture) pour les comparer au bloc APRÈS.

1. Vérifier que les clients existent déjà dans dim_customer

```
SELECT customer_id, customer_key
FROM dim_customer
WHERE customer_id IN (
  '82e136de4a3b5e97a17fb25fec7efc0a9de48b8727308bf734edee7fbad43e13',
  '306b509665ca96700fc7d6fb96c914ae28a860801320faa97dcbc0c8fc9948c4',
  'dfaf1056b112f0d796c0181d5cd6d5f9de7a781dcee597d2e3a6476e1a517128',
  '554b16516950f40b73e6b49f8ac2ffca177dd363f3a27bd14c7c9a0eb92d6f57',
  '9fc68991022633912f8ae236f388f39b6311ca3ef2cc1041d93999c4ba4cf784'
);
```

2. Vérifier que les articles existent déjà dans dim_article

```
SELECT article_id, article_key
FROM dim_article
WHERE article_id IN (
  '0740723001', '0656348001', '0603584008', '0614714004'
```

```
-- , '<5e article_id à compléter>'
);
```

3. Vérifier que la date testée n'existe pas encore dans dim_date

```
SELECT date, date_key, month, year
FROM dim_date
WHERE date = '2026-07-21';
-- Résultat attendu AVANT injection : 0 ligne (date encore jamais vue)
```

4. Nombre total de lignes dans fact_transaction (référence globale)

```
SELECT COUNT(*) AS nb_lignes_avant FROM fact_transaction;
```

5. Ventes agrégées du jour testé, avant injection

```
SELECT chiffre_affaires, nb_transactions, date_key
FROM daily_sales
WHERE date_key = (SELECT date_key FROM dim_date WHERE date = '2026-07-21');
-- Résultat attendu AVANT injection : 0 ligne (date_key = NULL, date inexistante)
```

6. Features clients avant recalcul

```
SELECT dc.customer_id, cft.*
FROM customers_features_train cft
JOIN dim_customer dc ON dc.customer_key = cft.customer_key
WHERE dc.customer_id IN (
    '82e136de4a3b5e97a17fb25fec7efc0a9de48b8727308bf734edee7fbad43e13',
    '306b509665ca96700fc7d6fb96c914ae28a860801320faa97dcbc0c8fc9948c4',
    'dfaf1056b112f0d796c0181d5cd6d5f9de7a781dcee597d2e3a6476e1a517128',
    '554b16516950f40b73e6b49f8ac2ffca177dd363f3a27bd14c7c9a0eb92d6f57',
    '9fc68991022633912f8ae236f388f39b6311ca3ef2cc1041d93999c4ba4cf784'
);
-- Noter précisément : total_spend, n_transactions, last_purchase,
-- recency_days et surtout segment_valeur (ancien segment) de chaque client
```

7. Segments concernés avant recalcul

```
SELECT *
FROM customer_segments_summary
WHERE segment_valeur IN (
    SELECT DISTINCT cft.segment_valeur
    FROM customers_features_train cft
    JOIN dim_customer dc ON dc.customer_key = cft.customer_key
    WHERE dc.customer_id IN (
        '82e136de4a3b5e97a17fb25fec7efc0a9de48b8727308bf734edee7fbad43e13',
        '306b509665ca96700fc7d6fb96c914ae28a860801320faa97dcbc0c8fc9948c4',
        'dfaf1056b112f0d796c0181d5cd6d5f9de7a781dcee597d2e3a6476e1a517128',
        '554b16516950f40b73e6b49f8ac2ffca177dd363f3a27bd14c7c9a0eb92d6f57',
        '9fc68991022633912f8ae236f388f39b6311ca3ef2cc1041d93999c4ba4cf784'
    )
);
-- Noter les noms exacts des segments obtenus : ils serviront à requêter
-- explicitement l'ANCIEN segment dans le bloc APRÈS, même si un client
-- en est parti (voir requête 5 du bloc APRÈS)
```

8. Historique brut des transactions des clients concernés (vérité terrain)

```
SELECT dc.customer_id, ft.price, ft.date_key, dd.date
FROM fact_transaction ft
JOIN dim_customer dc ON dc.customer_key = ft.customer_key
JOIN dim_date dd ON dd.date_key = ft.date_key
```



```

WHERE dc.customer_id IN (
  '82e136de4a3b5e97a17fb25fec7efc0a9de48b8727308bf734edee7fbad43e13',
  '306b509665ca96700fc7d6fb96c914ae28a860801320faa97dcbc0c8fc9948c4',
  'dfaf1056b112f0d796c0181d5cd6d5f9de7a781dcee597d2e3a6476e1a517128',
  '554b16516950f40b73e6b49f8ac2ffca177dd363f3a27bd14c7c9a0eb92d6f57',
  '9fc68991022633912f8ae236f388f39b6311ca3ef2cc1041d93999c4ba4cf784'
)
ORDER BY dc.customer_id, dd.date;

```

9. products_performance avant injection

```

SELECT *
FROM products_performance
WHERE article_key IN (
  SELECT article_key FROM dim_article
  WHERE article_id IN (
    '0740723001','0656348001','0603584008','0614714004'
    -- , '<5e article_id à compléter>'
  )
);
-- Si un article est vendu pour la 1ère fois : 0 ligne, c'est normal
-- (test d'INSERT et non d'UPDATE, comme pour daily_sales)

```

10. Vérité terrain des ventes par article, avant injection

```

SELECT da.article_id, da.article_key, COUNT(*) AS n_sales_recalcule
FROM fact_transaction ft
JOIN dim_article da ON da.article_key = ft.article_key
WHERE da.article_id IN (
  '0740723001','0656348001','0603584008','0614714004'
  -- , '<5e article_id à compléter>'
)
GROUP BY da.article_id, da.article_key;

```

BLOC 2 — REQUÊTES "APRÈS" (à exécuter une fois l'injection + le recalcul terminés)

À copier-coller et exécuter intégralement après l'injection des nouvelles transactions et une fois le job de recalcul terminé. Comparer chaque résultat à son équivalent du bloc AVANT.

1. Vérifier que dim_date a bien créé la nouvelle date

```

SELECT date, date_key, month, year
FROM dim_date
WHERE date = '2026-07-21';
-- Résultat attendu APRÈS injection : exactement 1 ligne, month = 7, year = 2026

```

2. Nombre total de lignes dans fact_transaction, après injection

```

SELECT COUNT(*) AS nb_lignes_apres FROM fact_transaction;
-- Attendu = nb_lignes_avant + nombre de transactions réellement injectées (5)

```

3. Ventes agrégées du jour testé, après injection, comparées à l'attendu

```

SELECT
  chiffre_affaires,
  0.090672 AS ca_attendu,      -- 🛠 somme des price injectés à recalculer à chaque
test
  nb_transactions,
  5 AS nb_attendu              -- 🛠 nombre de transactions réellement injectées

```

```
FROM daily_sales
WHERE date_key = (SELECT date_key FROM dim_date WHERE date = '2026-07-21');
-- Un écart (ou 0 ligne) signale un problème d'INSERT/UPSERT sur une
-- date encore jamais vue par le pipeline de recalcul.
```

4. Features clients après recalcul

```
SELECT dc.customer_id, cft.*
FROM customers_features_train cft
JOIN dim_customer dc ON dc.customer_key = cft.customer_key
WHERE dc.customer_id IN (
  '82e136de4a3b5e97a17fb25fec7efc0a9de48b8727308bf734edee7fbad43e13',
  '306b509665ca96700fc7d6fb96c914ae28a860801320faa97dcbc0c8fc9948c4',
  'dfaf1056b112f0d796c0181d5cd6d5f9de7a781dcee597d2e3a6476e1a517128',
  '554b16516950f40b73e6b49f8ac2ffca177dd363f3a27bd14c7c9a0eb92d6f57',
  '9fc68991022633912f8ae236f388f39b6311ca3ef2cc1041d93999c4ba4cf784'
);
-- Comparer à la requête 6 du bloc AVANT : total_spend doit augmenter du
-- price injecté, n_transactions +1, last_purchase = 2026-07-21,
-- recency_days = 0. Noter aussi le NOUVEAU segment_valeur de chaque client.
```

5. Segments concernés après recalcul (ANCIENS et NOUVEAUX, explicitement)

```
-- ⚠ Requête les segments en dur (ceux notés à l'étape 7 du bloc AVANT
-- ET les nouveaux relevés à l'étape 4 ci-dessus), pas seulement le
-- segment_valeur courant des clients : un client qui a changé de segment
-- doit être vérifié des DEUX côtés (perte dans l'ancien, gain dans le nouveau).
SELECT *
FROM customer_segments_summary
WHERE segment_valeur IN (
  '<ancien_segment_1>', '<ancien_segment_2>', ... ,
  '<nouveau_segment_1>', '<nouveau_segment_2>', ...
-- 🛠 remplacer par les segments exacts relevés avant/après
);
```

6. Recalcul indépendant des features clients depuis le fait (vérité terrain)

```
SELECT
  dc.customer_id,
  SUM(ft.price) AS total_spend_recalcule,
  COUNT(*) AS n_transactions_recalcule,
  MIN(dd.date) AS first_purchase_recalcule,
  MAX(dd.date) AS last_purchase_recalcule,
  COUNT(DISTINCT da.product_group_name) AS n_distinct_categories_recalcule
FROM fact_transaction ft
JOIN dim_customer dc ON dc.customer_key = ft.customer_key
JOIN dim_date dd ON dd.date_key = ft.date_key
JOIN dim_article da ON da.article_key = ft.article_key
WHERE dc.customer_id IN (
  '82e136de4a3b5e97a17fb25fec7efc0a9de48b8727308bf734edee7fbad43e13',
  '306b509665ca96700fc7d6fb96c914ae28a860801320faa97dcbc0c8fc9948c4',
  'dfaf1056b112f0d796c0181d5cd6d5f9de7a781dcee597d2e3a6476e1a517128',
  '554b16516950f40b73e6b49f8ac2ffca177dd363f3a27bd14c7c9a0eb92d6f57',
  '9fc68991022633912f8ae236f388f39b6311ca3ef2cc1041d93999c4ba4cf784'
)
GROUP BY dc.customer_id;
-- Comparer chaque ligne à la requête 4 ci-dessus (customers_features_train).
-- Tout écart = le job de recalcul ne prend pas en compte le streaming.
```

7. Cohérence globale fait ↔ features

```
SELECT
  (SELECT COUNT(*) FROM fact_transaction) AS total_fact,
  (SELECT SUM(n_transactions) FROM customers_features_train) AS total_features;
-- Les deux totaux doivent évoluer du même delta entre avant et après.
```

8. Recalcul indépendant des agrégats de segments

```
SELECT
  cft.segment_valeur,
  COUNT(*) AS nb_clients_recalcule,
  AVG(cft.total_spend) AS montant_moyen_recalcule,
  AVG(cft.n_transactions) AS achats_moyen_recalcule,
  SUM(cft.total_spend) AS ca_segment_recalcule
FROM customers_features_train cft
GROUP BY cft.segment_valeur;
-- Comparer segment par segment avec customer_segments_summary (requête 5).
```

9. products_performance après injection

```
SELECT *
FROM products_performance
WHERE article_key IN (
  SELECT article_key FROM dim_article
  WHERE article_id IN (
    '0740723001', '0656348001', '0603584008', '0614714004'
    -- , '<5e article_id à compléter>'
  )
);
-- n_sales attendu = valeur "avant" + 1 par transaction (ou création de
-- la ligne si l'article n'avait encore jamais été vendu)
```

10. Vérité terrain des ventes par article, après injection

```
SELECT da.article_id, da.article_key, COUNT(*) AS n_sales_recalcule
FROM fact_transaction ft
JOIN dim_article da ON da.article_key = ft.article_key
WHERE da.article_id IN (
  '0740723001', '0656348001', '0603584008', '0614714004'
  -- , '<5e article_id à compléter>'
)
GROUP BY da.article_id, da.article_key;
```

11. Delta direct products_performance vs vérité terrain (le vrai test)

```
SELECT
  da.article_id,
  pp.n_sales AS n_sales_table,
  recalc.n_sales_recalcule,
  pp.n_sales - recalc.n_sales_recalcule AS ecart
FROM dim_article da
LEFT JOIN products_performance pp ON pp.article_key = da.article_key
LEFT JOIN (
  SELECT article_key, COUNT(*) AS n_sales_recalcule
  FROM fact_transaction
  GROUP BY article_key
) recalc ON recalc.article_key = da.article_key
WHERE da.article_id IN (
  '0740723001', '0656348001', '0603584008', '0614714004'
  -- , '<5e article_id à compléter>'
)
```

```
);  
-- ecart doit être 0 pour chaque article. Le LEFT JOIN capture aussi le  
-- cas où pp.n_sales serait NULL (ligne pas encore créée).  
-- Vérifier également que prod_name, product_group_name, department_name  
-- et index_name restent identiques avant/après (colonnes descriptives  
-- issues de dim_article, jamais recalculées).
```

Checklist finale de validation

- ☐ dim_date crée automatiquement la nouvelle date (month/year corrects), sans intervention manuelle
- ☐ Le nombre de lignes fact_transaction augmente exactement du nombre de transactions injectées (0 doublon, 0 perte)
- ☐ daily_sales gère correctement le cas d'une nouvelle ligne (INSERT), pas seulement une mise à jour (UPDATE/UPSERT)
- ☐ customers_features_train est recalculé pour chaque client impacté (total_spend, n_transactions, last_purchase, recency_days)
- ☐ Aucun client non concerné n'a été modifié par erreur
- ☐ Si un client change de segment_valeur : il disparaît bien de son ancien segment ET apparaît dans le nouveau, avec nb_clients et ca_segment à jour des deux côtés
- ☐ products_performance est mis à jour (ou créé) pour chaque article vendu, avec le bon n_sales et sans altération des colonnes descriptives
- ☐ Latence mesurée entre l'injection streaming et la réplication du recalcul dans les tables agrégées